28th March, Authentication and Authorization

— Jaani

1

⇒ Passwords are generally hashed, so to chash passwords, we use middlewares.

⇒ Types of middlewares (based on timing)

1. Pre Middleware (runs before handler/main logic)
2. Post Middleware (runs after response logic) mostly for cleanup.

⇒ userSchema.pre ('save', async fun(~~candidate Password~~ next){

```
~~if (this.password~~
if (!this.isModified("password")) return next();
this.password = await bcrypt.hash(this.password, 12)
next();
});
```

no of salt rounds (more rounds mean more secure)

1. Bcrypt vs Bcryptjs

Bcrypt → mostly used in building production apps

Bcryptjs → mostly used for learning.

To verify password,
we need to convert the password recieved from user using salt and check if same, then the user is correct or verified.

⇒

```
userSchema.methods.comparePassword = (Async fun
(clearTextPassword) {
return bcrypt.compare(clearTextPassword, this.password);
});
```

← the pass user typed during login

↑ pass in DB

This checks if the password user entered match the hashed pass in DB. Here bcrypt handles salt + hashing internally during compare.

⇒ gives boolean value

const isMatch = await user.comparePassword(password);

if not found/match.

2

gives Error also

if (!user.isverified) {

throw ApiError.forbidden("Pls verify your email before login");

⇒ We want to make a middleware of auth name isLoggedIn so we don't have to repeat code, we will just write "isLoggedIn" that will make sure user is authenticated.

In auth.middleware.js

⇒ const authenticate = async (req, res, next) ⇒ {

let token;

req.headers.authorization ?.startsWith("Bearer"

⇒ We want the store the accesstoken at the user side in "httpOnly" (Cookie)

⇒ Ways to send token ⇒

1. JSON response
2. HTTP only Cookie
3. How client sends it back:
   ⇒ In headers

   Authorization: Bearer <token>

⇒ Code continued →

⇒ if (req.headers.authorization ?.startsWith("Bearer")){

token = req.headers.authorization.split(" ")[1]

}

if (!token) throw ApiError.unauthorized("Not Authenticated")

const ⇒ verifyAccessToken(token)

decoded

const user = await User.findById(decodedId)

continued ⇒

```
const user = await User.findById(decoded.id)
  if (!user) throw ApiError.unauthorized("User no longer exists")
  req.user = {
      id : user._id,
      role : user.role,
      name : user.name,
      email : user.email,
  };
  next();
};
```

Does roles array

```
const authorize = (...roles) => {
    return (req, res, next) => {
        if (!roles.includes(req.user.role)){
        throw ApiError.forbidden(
        "You do not have Permission to perform this action",
        );
    } next();
}
```

This middleware allows only specific roles to access a route, otherwise throws a forbidden error.

⇒ In auth.routes.js

```
router.post("/login", validate(LoginDto), controller.login);
router.get("/me"), authenticate, controller.getMe);
router.get("/verify-email/:token", controller.getMe);
```

⇒ create new file in dto folder name login.dto.js

Inside login.dto.js

```
import Joi from "joi"

class LoginDto extends BaseDto{
  static schema = Joi.object({
    email : Joi.string().email().lowerCase().required(),
    password : Joi.string().required(),
  });
}
```

Inside controller, auth.controller.js

```
const login = async (req, res) => {
const {user=await authservice.login(req.body)
  accesToken
  refreshToken
  }
  res.cookie("refreshToken", refreshToken,
    {
    httpOnly: true,
    maxAge : 7 * 24 * 60 * 1000 (for 7 days)
    secure : true,
    });
  ApiResponse.ok(res, "Login successfull", {user,
                                  accesToken})
};

const logout = async (req, res) => {
    authservice.Logout(req.user.id)
    res.clearCookie("refreshToken")
    ApiResponse.ok(res, "Logout success");
};
```

npm i cookie Parser

These are options

Add a route also

```
router.post("/logout", authenticate, controller.logout);
```

Inside auth.service.js

```
const getMe = async(userId) => {
    const user = await User.findById(userId)
    if (!user) throw ApiError.notFound("User Not Found")
    return user;
};
```

Now write controller for same.

Inside auth.controller.js

```
const getMe = async(req, res) => {
    const user = await authservice.getMe(req.user.id);
    ApiResponse.ok(res, "User Profile", user);
};
```

Now we start making a common service for sending mail using NodeMailer (We use mailtrap for testing)

⇒ Inside common ⇒ config ⇒ email.js

Inside email.js

```
import nodemailer from "nodemailer"
const transporter = nodemailer.createTransport({
    . . . . . .
};
try {
    . . .
Catch(){
}
```

copy from nodemailer website

Or, we can create a basic method sendMail

```
const sendMail = async (to, subject, html) => {
  await transporter.sendMail({
    from : `${process.env.SMTP_FROM_EMAIL}`,
    to,
    subject,
    html,
  })
}
```

To make verification Email ;

```
const sendVerificationEmail = async (email, token) => {
  await transporter.sendMail({
    from : `${process.env.SMTP_FROM_EMAIL}`,
    email,
    subject,
    html,
  })
}
```

can make custom template

Now you can use to send this email, you can do it (like add in service)

To verify Email :

```
const verifyEmail = async (token) => {
  const hashedToken = hashToken(token);
  const user = await User.findOne({ verificationToken : hashedToken})
    .select("+verificationToken")
);
```

```
// if user not found
user.isVerified = true
user.verificationToken = undefined
await user.save()
return user;
```

Now in app.js

```
import express from "express"
import cookieParser from "cookie-parser"
import authRoute from "./modules/auth/auth.routes.js"

const app = express()
app.use(express.json());
app.use(express.urlencoded({extended: true}));
app.use(cookieParser());
app.use('/api/auth', authRoute);
export default app;
```

Assignment : study more about salt in cryptography.

" next() is a fun in Express middleware, and it passes control to next middleware or route handler in the request-response cycle. Next is a callback.

Happy Learning!

Can follow me on X: jaaaani404